

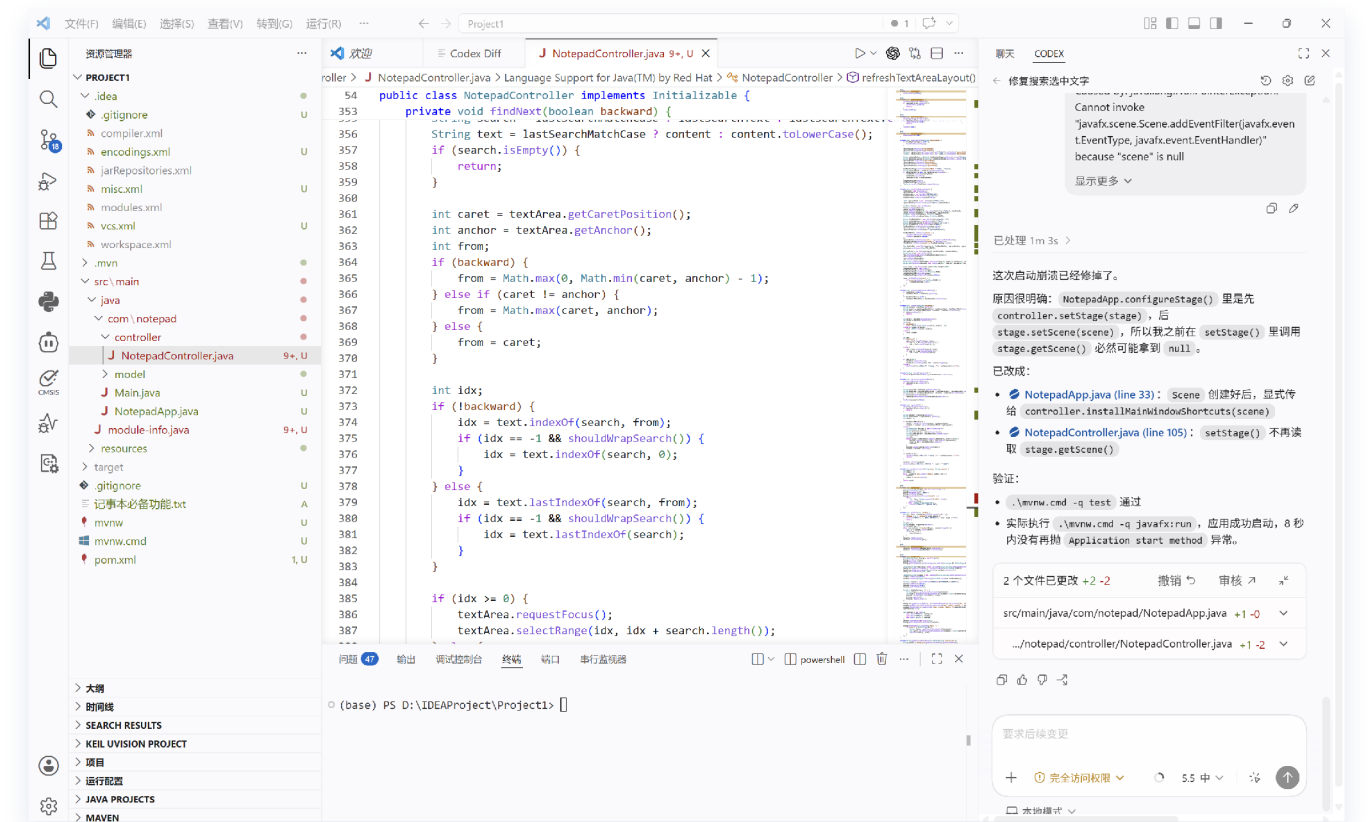
AI辅助编程记事本项目

借助AI过程

生成计划



完成开发



使用总结

一、AI擅长解决的问题

- 1. 项目规划与架构设计 AI能快速给出完整的项目结构、技术选型和分层设计（如MVC架构），节省了大量前期调研时间。本次开发中，从需求文档到完整的开发计划（包含pom.xml、FXML布局、Controller方法签名）几乎一次生成。
- 2. 样板代码生成 菜单栏、文件读写、对话框等重复性代码，AI能快速补全，避免查阅大量文档。JavaFX的FXML结构、FileChooser用法、Alert对话框等，AI均能直接给出可运行的代码。
- 3. Bug定位与修复 描述报错信息后，AI能快速分析原因并给出修改方案，比自己逐行排查效率更高。

二、提示词技巧

场景	效果差的写法	效果好的写法
生成代码	"帮我写记事本"	"用JavaFX 21 + Maven，实现Windows记事本的查找替换功能，要求非模态对话框，含大小写选项"

场景	效果差的写法	效果好的写法
修复Bug	"代码报错了"	"运行报 NullPointerException，在 NotepadController 第 58 行，附上完整错误栈"
规划任务	"帮我规划一下"	"我要把计划发给Codex执行，请输出包含完整代码片段的可执行计划，分优先级"

核心原则：说清楚技术栈版本、具体场景、输出格式要求，AI的回答会更精准。

三、模型分工策略

本次项目采用了codex进行代码的完成并使用了差异化的模型使用策略，实践下来效果不错：

- **GPT5.4思考 Low**：负责主干功能开发。任务明确、重复性高的代码生成，低思考档就够用，速度快、成本低。
- **GPT5.5 思考Mid**：专门用于Bug修复。Bug排查需要更强的推理能力，值得用更高档次的思考。

这个经验说明：不同任务匹配不同思考强度，既能保证质量，又避免浪费。

四、整体感受

本次开发基本没有遇到明显困难，主要原因有：

1. 提示词写得比较具体，AI生成的内容可直接使用
2. 任务拆分合理，主干和Bug修复分开处理
3. 对JavaFX有基础了解，能判断AI输出是否合理

不足之处：AI偶尔生成的代码需要小幅调整（如路径写法、版本兼容），说明使用AI时仍需要具备一定的基础知识来做验证。

运行截图

